

Proyecto de Machine Learning

Predicción de posiciones en Fórmula 1 usando aprendizaje supervisado

Reporte técnico

Dataset: Formula 1 World Championship (1950-2024)

Modelos: Red neuronal artificial y Gradient Boosting Regressor

Contenido del documento

- Introducción
- Dataset
- Preprocesamiento y feature engineering
- División de datos
- Modelos implementados
- Experimentos
- Análisis de gráficas
- Resultados
- Conclusión
- Trabajo futuro
- Referencias

Introducción

La Fórmula 1 es un deporte altamente basado en datos, donde el resultado de una carrera puede depender de factores como el piloto, el constructor, el circuito, la posición de salida, el rendimiento histórico y el contexto de la temporada. Debido a esta gran cantidad de variables, los datos históricos de Fórmula 1 pueden utilizarse para resolver problemas de aprendizaje supervisado.

El objetivo de este proyecto es implementar y evaluar modelos de aprendizaje automático capaces de predecir la posición final de un piloto en una carrera de Fórmula 1. La variable objetivo utilizada fue `positionOrder`, la cual representa la posición final obtenida por un piloto en una carrera. Debido a que esta variable es numérica, el problema se plantea como una tarea de regresión.

El proyecto toma como referencia metodológica el trabajo de O'Hanlon, donde se utilizan modelos de aprendizaje supervisado, incluyendo redes neuronales artificiales, para predecir rankings de Fórmula 1. A partir de esta base, el proyecto evolucionó desde una red neuronal inicial con pocas variables hacia una comparación más completa entre una red neuronal profunda y un modelo de Gradient Boosting Regressor, utilizando variables históricas y contextuales generadas mediante feature engineering.

Dataset

El dataset utilizado en este proyecto es Formula 1 World Championship (1950-2024), disponible en Kaggle. Este conjunto de datos contiene información histórica de Fórmula 1 organizada en varios archivos CSV relacionados entre sí.

En la versión inicial del proyecto se utilizaron únicamente archivos principales como `results.csv`, `racers.csv`, `drivers.csv`, `constructors.csv` y `circuits.csv`. Sin embargo, para la versión final se amplió el proceso de preprocesamiento mediante la incorporación de archivos adicionales, con el objetivo de enriquecer la información disponible para cada piloto en cada carrera.

Los principales archivos utilizados fueron:

- `results.csv`: contiene los resultados de carrera, posición final, posición de salida, vueltas completadas, puntos e identificadores del piloto y constructor.
- `racers.csv`: contiene información de cada carrera, como año, ronda, fecha e identificador del circuito.
- `drivers.csv`: contiene información de los pilotos, incluyendo fecha de nacimiento y referencia del piloto.
- `constructors.csv`: contiene información de los equipos o constructores.
- `circuits.csv`: contiene información de los circuitos y países.
- `qualifying.csv`: contiene información de clasificación, utilizada para obtener la posición de clasificación del piloto.
- `driver_standings.csv`: contiene información del campeonato de pilotos antes o durante la temporada.
- `constructor_standings.csv`: contiene información del campeonato de constructores.
- `status.csv`: contiene información relacionada con el estado final del piloto en la carrera, como finalización, abandono o fallas.

La variable objetivo seleccionada fue `positionOrder`, es decir, la posición final del piloto en la carrera. Para reducir inconsistencias históricas causadas por cambios en reglamentos, formatos de carrera, sistemas de puntuación y diferencias entre épocas, se filtraron únicamente carreras desde el año 2010 en adelante.

Preprocesamiento y feature engineering

El preprocesamiento de los datos incluyó la carga, selección y unión de múltiples archivos CSV mediante identificadores comunes como `raceId`, `driverId`, `constructorId` y `circuitId`. El objetivo fue construir una tabla final donde cada fila representa la participación de un piloto en una carrera.

Los pasos principales del preprocesamiento fueron:

1. Carga de los archivos CSV necesarios.
2. Selección de columnas relevantes de cada archivo.
3. Unión de los datasets mediante identificadores comunes.
4. Filtrado de carreras desde el año 2010 en adelante.
5. Eliminación o tratamiento de registros inválidos, como posiciones de salida igual a cero cuando no eran útiles para el modelo.
6. Definición de positionOrder como variable objetivo.
7. Generación de variables históricas y contextuales.
8. Codificación de variables categóricas mediante one-hot encoding.
9. Separación del dataset 80/20 para train/test.
10. Escalamiento de variables mediante StandardScaler.
11. Entrenamiento y evaluación de modelos de regresión.

Las variables categóricas, como piloto, constructor, circuito y país, fueron transformadas a formato numérico mediante one-hot encoding, ya que los modelos de aprendizaje automático no pueden trabajar directamente con texto. Además, se aplicó escalamiento a las variables numéricas para mejorar el entrenamiento de la red neuronal, ya que las redes neuronales son sensibles a la escala de entrada.

Variables base

Las primeras variables utilizadas estaban relacionadas con información directa de la carrera:

- year
- round
- grid
- qualifying_position
- country
- driverRef
- constructorRef
- circuitRef

Estas variables permiten representar el contexto básico de cada carrera, incluyendo la temporada, la ronda, la posición de salida, el piloto, el equipo y el circuito.

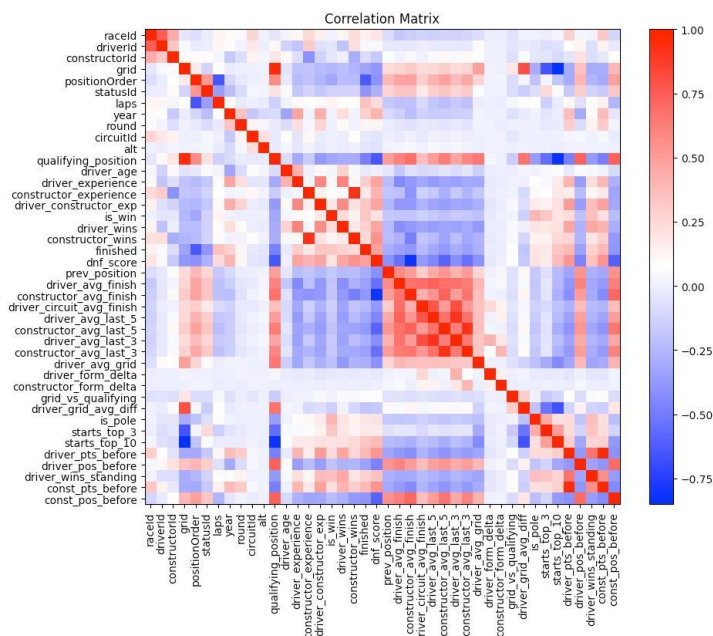


imagen matriz de correlación de variables del dataset.

Variables históricas

Para mejorar el desempeño del modelo, se agregaron variables históricas calculadas a partir de información previa. Estas variables permiten representar el rendimiento pasado del piloto, el constructor y la combinación piloto-equipo.

Algunas de las variables generadas fueron:

- driver_age
- driver_experience
- constructor_experience
- driver_constructor_exp
- driver_wins
- constructor_wins
- prev_position
- driver_avg_finish
- constructor_avg_finish
- driver_circuit_avg_finish
- driver_avg_last_5
- constructor_avg_last_5
- driver_form_delta
- constructor_form_delta

Estas variables fueron importantes porque permitieron al modelo capturar patrones de rendimiento acumulado y forma reciente. Por ejemplo, driver_avg_last_5 representa el rendimiento promedio reciente del piloto, mientras que driver_form_delta ayuda a identificar si el piloto está mejorando o empeorando respecto a su rendimiento histórico.

Variables de campeonato

También se incorporaron variables relacionadas con el estado del campeonato antes de la carrera:

- driver_pts_before
- driver_pos_before
- driver_wins_standing
- const_pts_before
- const_pos_before

Estas variables permiten representar el nivel competitivo del piloto y del constructor dentro de la temporada. Un piloto con más puntos o mejor posición en el campeonato probablemente pertenece a un contexto competitivo más fuerte, lo cual puede influir en su posición final.

Variables derivadas de clasificación y salida

Además, se generaron variables relacionadas con la posición de salida y clasificación:

- grid_vs_qualifying
- is_pole
- starts_top_3
- starts_top_10

Estas variables ayudan a representar el impacto de la clasificación en el resultado final. Por ejemplo, iniciar en pole position o dentro de los primeros tres lugares suele estar asociado con mejores probabilidades de terminar en posiciones delanteras.

División de datos

Para la evaluación final del proyecto se utilizó una división aleatoria del dataset, con una proporción de 80% para entrenamiento y 20% para prueba. Esta estrategia permitió entrenar y evaluar los modelos con una muestra amplia y variada de carreras, pilotos, constructores y circuitos disponibles en el dataset.

El uso de un split aleatorio permitió comparar de manera clara el desempeño de la red neuronal artificial y del modelo Gradient Boosting Regressor bajo las mismas condiciones. Además, facilitó la generación de visualizaciones interpretables, como las gráficas de posiciones reales contra posiciones predichas.

Durante el desarrollo también se realizaron pruebas con una división temporal, entrenando con temporadas anteriores y evaluando con temporadas más recientes. Este enfoque representa una tarea más estricta, ya que simula la predicción de carreras futuras. Sin embargo, redujo considerablemente el rendimiento del modelo y produjo visualizaciones menos claras. Por esta razón, el split temporal se consideró como análisis exploratorio, mientras que la evaluación final del proyecto se realizó con el split aleatorio 80/20.

Es importante mencionar que el split aleatorio puede producir resultados más optimistas que un split temporal, ya que mezcla información de distintas temporadas entre entrenamiento y prueba. Sin embargo, para los objetivos de este proyecto académico, este enfoque permitió evaluar la capacidad general de los modelos para aprender patrones históricos y comparar de forma directa diferentes algoritmos de regresión.

Modelos implementados

Red neuronal artificial

El primer enfoque principal fue una red neuronal artificial para regresión. La red recibe las variables preprocesadas como entrada y produce un valor numérico correspondiente a la posición final estimada del piloto.

La arquitectura inicial fue sencilla:

- Capa densa de 32 neuronas con activación ReLU.
- Capa densa de 16 neuronas con activación ReLU.
- Capa de salida de una neurona con activación lineal.

Posteriormente, se probaron arquitecturas más amplias para mejorar el desempeño. La mejor arquitectura de red neuronal utilizada fue:

- Capa densa de 128 neuronas con activación ReLU y regularización L2.
- Dropout de 0.2.
- Capa densa de 64 neuronas con activación ReLU.
- Dropout de 0.2.
- Capa densa de 32 neuronas con activación ReLU.
- Capa de salida de una neurona con activación lineal.

La función de activación lineal en la capa de salida se utilizó porque el problema es de regresión. El modelo produce un valor numérico, no una clase.

El modelo fue compilado con:

- Optimizador: Adam.
- Función de pérdida: Mean Squared Error.
- Métrica durante entrenamiento: Mean Absolute Error.

Además, se agregaron callbacks para mejorar el proceso de entrenamiento:

- EarlyStopping: detiene el entrenamiento cuando la pérdida de validación deja de mejorar.
- ReduceLROnPlateau: reduce la tasa de aprendizaje cuando el modelo se estanca.

Estos callbacks permitieron controlar el sobreentrenamiento y conservar los mejores pesos obtenidos durante el entrenamiento.

Gradient Boosting Regressor

Además de la red neuronal, se implementó un modelo GradientBoostingRegressor como comparación. Este tipo de modelo pertenece a la familia de métodos basados en árboles y suele tener buen desempeño en datos tabulares.

El Gradient Boosting construye múltiples árboles de decisión de forma secuencial, donde cada nuevo árbol intenta corregir los errores cometidos por los anteriores. Esto permite capturar relaciones no lineales entre las variables sin requerir el mismo tipo de escalamiento o arquitectura que una red neuronal.

La inclusión de este modelo permitió comparar si una red neuronal era realmente la mejor opción para este dataset o si un método clásico de machine learning podía obtener un mejor desempeño.

Experimentos

Experimento 1: red neuronal inicial

El primer modelo utilizó una arquitectura simple de red neuronal con pocas variables predictoras y una división aleatoria del dataset. Este experimento obtuvo un rendimiento alto, con un R^2 cercano a 0.87. Sin embargo, posteriormente se identificó que esta evaluación podía ser demasiado optimista debido al uso de una división aleatoria y a una menor cantidad de controles sobre variables históricas y temporales.

Este experimento sirvió como punto de partida para comprobar que el dataset contenía información útil para predecir la posición final.

Experimento 2: split temporal

Después se cambió la evaluación a una división temporal. Esto redujo el rendimiento del modelo, pero permitió una evaluación más realista. La caída en el rendimiento mostró que predecir temporadas futuras es considerablemente más difícil que predecir datos mezclados aleatoriamente.

Este cambio fue importante porque el objetivo real del proyecto es predecir resultados futuros a partir de información pasada.

Experimento 3: feature engineering histórico

Se agregaron variables históricas relacionadas con rendimiento pasado, experiencia, victorias, posiciones anteriores, desempeño reciente y standings de pilotos y constructores.

La incorporación de estas variables mejoró notablemente el desempeño del modelo, ya que permitió capturar información más representativa del estado competitivo de cada piloto y equipo antes de una carrera.

Experimento 4: red neuronal mejorada

Se probaron diferentes arquitecturas de red neuronal. La arquitectura que mostró mejor desempeño fue una red con capas de 128, 64 y 32 neuronas, regularización L2, Dropout, EarlyStopping y ReduceLROnPlateau.

Este modelo alcanzó resultados competitivos, con valores de R^2 alrededor de 0.58 a 0.63 en las mejores ejecuciones. Debido a la aleatoriedad propia del entrenamiento de redes neuronales, el resultado exacto varió entre ejecuciones.

Experimento 5: Gradient Boosting Regressor

Finalmente, se comparó la red neuronal con un modelo GradientBoostingRegressor. Este modelo obtuvo el mejor desempeño general, superando ligeramente a la red neuronal.

Los resultados obtenidos por el Gradient Boosting Regressor fueron:

Modelo	MAE	RMSE	R ²
Red neuronal mejorada	2.89	3.75	0.58
Gradient Boosting Regressor	2.48	3.43	0.65

Tabla 1. Resultados obtenidos durante la comparación experimental de modelos.

Estos resultados indican que el modelo basado en árboles logró capturar mejor las relaciones presentes en el dataset tabular.

Experimento adicional: split temporal

También se realizó una prueba con división temporal, entrenando el modelo con temporadas anteriores y evaluándolo en temporadas más recientes. Este enfoque es más estricto porque simula la predicción de carreras futuras. Sin embargo, el rendimiento disminuyó considerablemente y la gráfica de posiciones reales contra predichas fue menos clara.

Por esta razón, el split temporal se reporta como análisis adicional, pero la evaluación final del proyecto se realizó con una división aleatoria 80/20.

Análisis de gráficas

Entrenamiento de la red neuronal

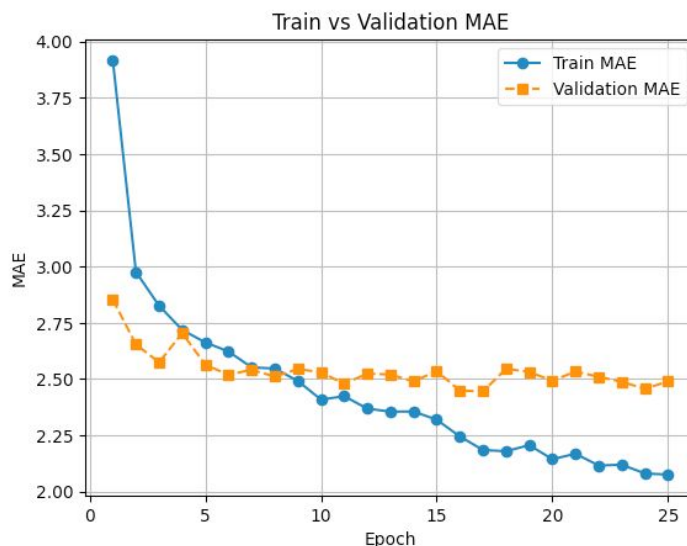


imagen comportamiento del MAE en entrenamiento y validación.

La gráfica de MAE muestra que el error de entrenamiento disminuye de forma constante durante las épocas, pasando de un valor cercano a 3.9 hasta aproximadamente 2.1. Esto indica que la red neuronal aprende progresivamente los patrones del conjunto de entrenamiento.

El MAE de validación también disminuye durante las primeras épocas, pero después se estabiliza alrededor de 2.5. Esta diferencia entre entrenamiento y validación indica que el modelo sigue mejorando en los datos de entrenamiento, pero su capacidad de mejora en datos no vistos se vuelve más limitada después de cierto punto. Esto puede interpretarse como una señal leve de sobreajuste, aunque el desempeño de validación se mantiene estable.

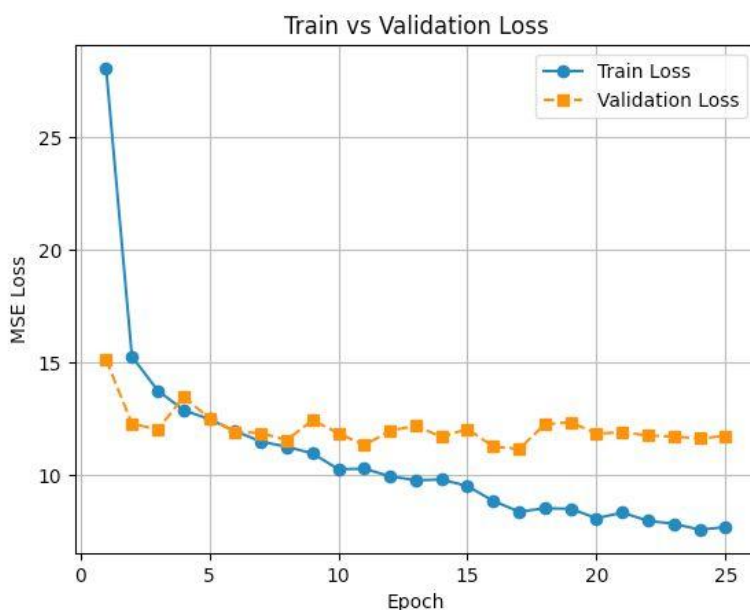


imagen train vs test loss.

La gráfica de pérdida muestra un comportamiento similar. La pérdida de entrenamiento disminuye constantemente, mientras que la pérdida de validación baja al inicio y luego se estabiliza alrededor de valores entre 11 y 12. Esto indica que el modelo aprende rápidamente en las primeras épocas, pero después alcanza un punto donde seguir entrenando ya no mejora de forma significativa la generalización.

El uso de EarlyStopping y ReduceLROnPlateau ayuda a controlar este comportamiento, ya que permite conservar los mejores pesos y ajustar la tasa de aprendizaje cuando el modelo deja de mejorar en validación.

Posiciones reales contra posiciones predichas

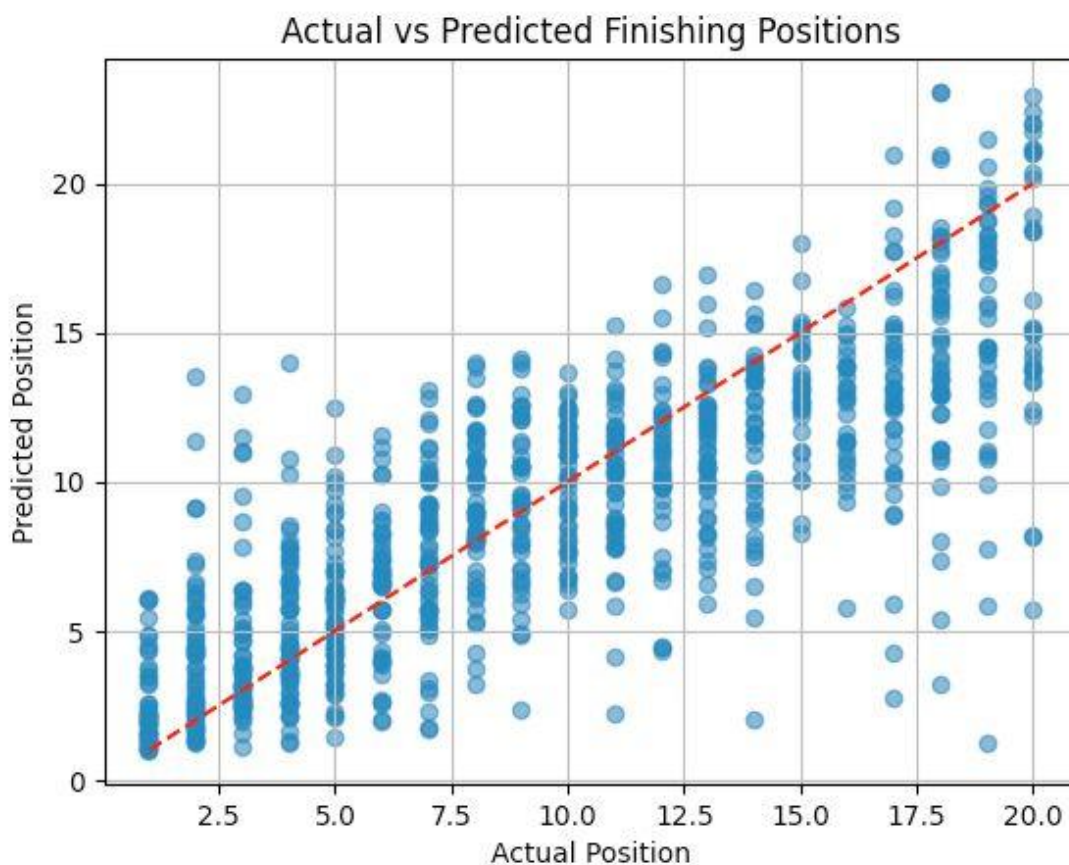


imagen posiciones reales contra posiciones predichas de la red neuronal.

La gráfica de posiciones reales contra posiciones predichas de la red neuronal muestra una tendencia positiva clara: conforme aumenta la posición real, también aumenta la posición predicha. Esto indica que el modelo sí logró aprender una relación general entre las variables de entrada y la posición final.

Sin embargo, también se observa dispersión alrededor de la línea diagonal. Esto significa que, aunque el modelo predice correctamente la tendencia general, existen errores en casos específicos. Las posiciones más difíciles de predecir suelen ser las posiciones medias y bajas, donde los resultados pueden verse afectados por eventos de carrera difíciles de anticipar.

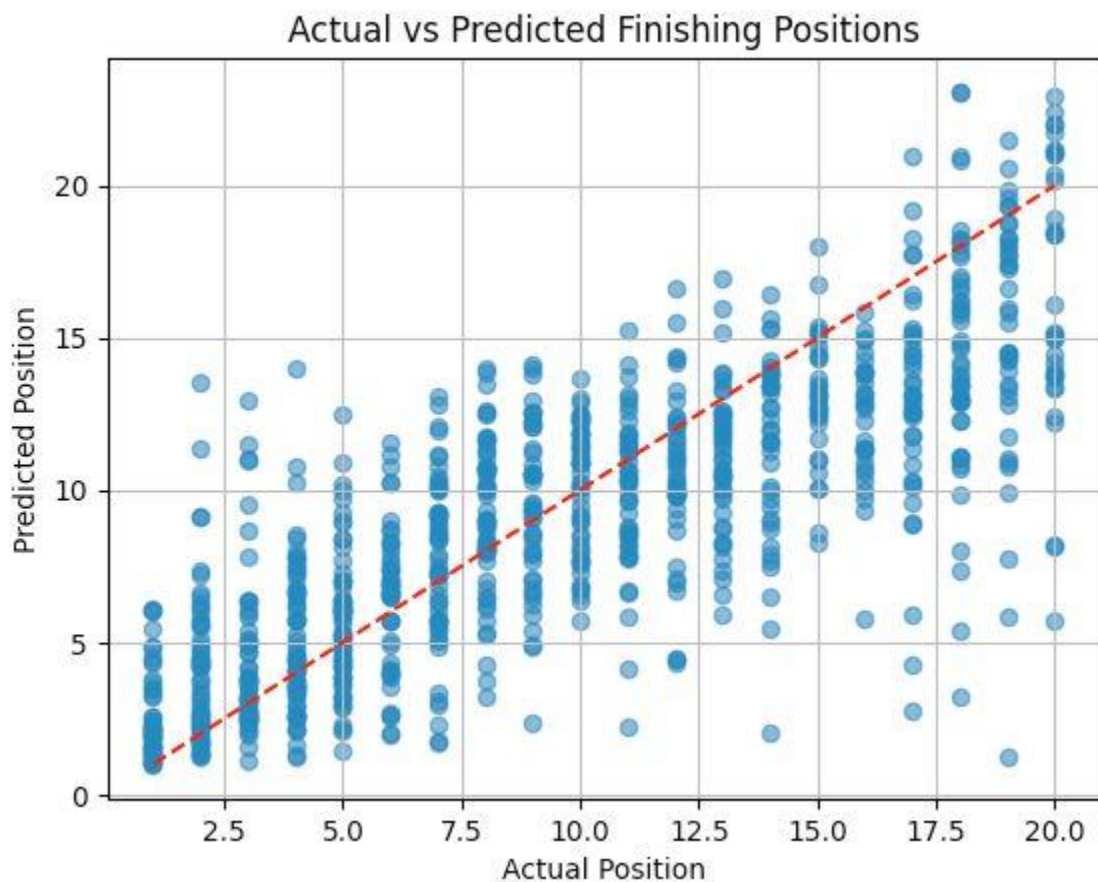


imagen actual vs predicted con Gradient Boosting Regressor.

La gráfica correspondiente al Gradient Boosting Regressor muestra una relación más ajustada entre posiciones reales y predichas. Los puntos se concentran con mayor claridad alrededor de la línea diagonal, lo cual coincide con sus mejores métricas de evaluación.

Aun así, existen valores alejados de la diagonal. Estos errores son esperados en el contexto de Fórmula 1, ya que la posición final no depende únicamente del rendimiento histórico o de la posición de salida, sino también de factores impredecibles como accidentes, fallas mecánicas, penalizaciones, estrategia, clima o safety cars.

En general, ambas gráficas muestran que los modelos capturan la tendencia principal del problema, pero el Gradient Boosting Regressor logra predicciones más consistentes.

Resultados

La evaluación final se realizó utilizando tres métricas principales de regresión:

- Mean Absolute Error.
- Root Mean Squared Error.
- R^2 score.

El Mean Absolute Error permite interpretar el error promedio en posiciones. Por ejemplo, un MAE de 2.29 indica que el modelo se equivoca, en promedio, por aproximadamente dos a tres posiciones al predecir el resultado final de un piloto.

El RMSE penaliza con mayor fuerza los errores grandes, por lo que es útil para identificar si existen predicciones muy alejadas del resultado real. En Fórmula 1, estos errores pueden aparecer cuando un piloto que normalmente tendría buen rendimiento termina en una posición baja debido a accidentes, fallas mecánicas, penalizaciones o abandonos.

El R^2 indica qué proporción de la variabilidad de la posición final logra explicar el modelo. Un R^2 más alto representa una mejor capacidad para capturar la relación entre las variables predictoras y la posición final.

Los resultados finales fueron los siguientes:

Modelo	MAE	RMSE	R^2
Improved Neural Network	2.56	3.48	0.62
Gradient Boosting Regressor	2.29	3.16	0.69

Tabla 2. Resultados finales de los modelos evaluados.

El modelo Gradient Boosting Regressor obtuvo el mejor desempeño general, con menor MAE, menor RMSE y mayor R^2 . Esto indica que el modelo basado en árboles logró capturar mejor las relaciones presentes en los datos tabulares.

La red neuronal artificial también obtuvo un desempeño competitivo, alcanzando un R^2 de 0.62. Sin embargo, su error fue mayor que el del Gradient Boosting Regressor. Esto sugiere que, aunque la red neuronal logró aprender patrones importantes del dataset, el modelo basado en árboles fue más efectivo para este tipo de problema.

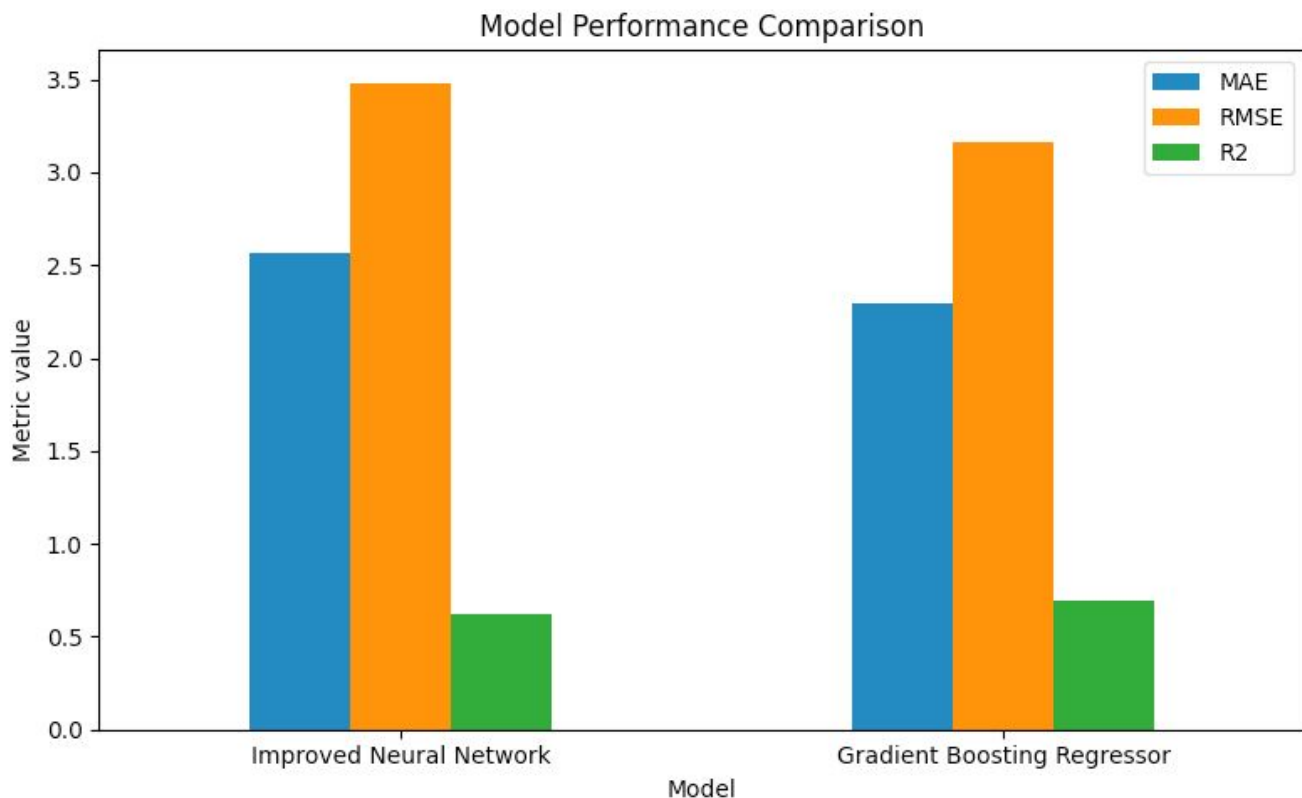


imagen comparación de modelos.

La gráfica de comparación de modelos muestra visualmente que el Gradient Boosting Regressor supera a la red neuronal en las tres métricas evaluadas. Su MAE y RMSE son menores, mientras que su R^2 es mayor. Esto permite concluir que, para este dataset tabular, el modelo de Gradient Boosting fue la mejor alternativa entre los modelos probados.

Conclusión

El proyecto logró desarrollar y evaluar modelos de aprendizaje supervisado para predecir posiciones finales en Fórmula 1. A lo largo del desarrollo se pasó de una red neuronal inicial con pocas variables a una versión más completa basada en múltiples archivos CSV, feature engineering histórico y comparación entre modelos.

La incorporación de variables históricas y contextuales fue fundamental para mejorar el desempeño. Estas variables permitieron representar mejor el rendimiento del piloto, el constructor, la posición de salida, la clasificación, el estado del campeonato y la forma reciente.

La red neuronal mejorada obtuvo resultados competitivos, alcanzando un MAE de 2.56, RMSE de 3.48 y R^2 de 0.62. Sin embargo, el mejor desempeño general fue obtenido por el Gradient Boosting Regressor, con un MAE de 2.29, RMSE de 3.16 y R^2 de 0.69.

Estos resultados muestran que, aunque una red neuronal puede aprender patrones útiles del dataset, un modelo basado en árboles puede ser más adecuado para este tipo de datos tabulares. El Gradient Boosting Regressor logró capturar mejor las relaciones no lineales entre variables históricas, posición de salida, clasificación, piloto, constructor y posición final.

La evaluación final se realizó con un split aleatorio 80/20, lo cual permitió comparar ambos modelos bajo las mismas condiciones y obtener visualizaciones claras del desempeño. También se realizaron pruebas con split temporal como análisis exploratorio, pero este enfoque representó una tarea más estricta y redujo considerablemente el rendimiento.

Aunque el modelo no puede predecir perfectamente eventos impredecibles como abandonos, accidentes, penalizaciones o fallas mecánicas, sí logra capturar una parte importante de la relación entre el rendimiento histórico y la posición final.

Trabajo futuro

Como trabajo futuro, se podrían explorar las siguientes mejoras:

- Incorporar información climática, penalizaciones, tipo de neumático o estrategia.
- Convertir la predicción de posición en un problema de clasificación por grupos, por ejemplo podio, puntos, media tabla o fondo.

Referencias

O'Hanlon, E. (2022). Using Supervised Machine Learning to Predict the Final Rankings of the 2021 Formula One Championship. National College of Ireland.

Kaggle. (s. f.). Formula 1 World Championship (1950-2024).